# Predictive Inventory & Workforce Planning System

## Project Requirements & Technical Design Document

*Product: EcoFit Smart Bottle | Duration: 6-10 Weeks | Team Size: 2*

Prepared for the IEEE Mentorship Program

*Includes: Data Dictionary, Methodology, ML Model Specification, and System Architecture*

# Table of Contents

# 1. Introduction & Document Purpose

This document is the technical requirements specification for the Predictive Inventory & Workforce Planning System project, to be delivered as 6-10 weeks mentored project by two student team members. It defines what the system must do, what data it will use, how each module should be built, which machine learning methods apply, and what "done" looks like for each deliverable.

The project uses three synthetic datasets — Sales, Inventory, and Workforce — purpose-built to behave like real operational data for a single consumer product ("EcoFit Smart Bottle") over a 2-year history (Jan 2023–Dec 2024). The data dictionary is present in this document and are described fully in Section 4 (Data Dictionary).

## 1.1 Background

Organizations that sell physical products face a recurring planning problem: customer demand fluctuates, and two expensive resources — inventory and labor — must be provisioned in advance to meet that demand without overspending. Traditional planning is often manual, reactive, and based on gut judgment rather than data, leading to stockouts, overstocking, understaffing, and excess labor cost.

This project builds a decision-support platform that forecasts future demand for a single product and translates that forecast into two parallel sets of recommendations: how much stock to hold and when to re-order, and how many workers are needed and when to hire. The platform does not automate decisions — it supports human decision-makers with forecasts, computed thresholds, and what-if scenarios.

## 1.2 Scope Simplification

To make the project achievable by two students, the scope is deliberately narrowed to a single product, single location model. This removes the complexity of cross-store and cross-SKU aggregation while preserving every core data science and modeling challenge — forecasting, reorder logic, and workforce estimation all remain fully present. This simplification decision is made to build a foundation that could be extended to multiple products in future iterations. This is not as a limitation on the depth of work expected.

# 2. Project Objectives

The project has six objectives. Objectives O1–O3 are the technical core and map directly to the three datasets and three functional modules. O4 and O5 integrate the technical modules into something a business user could actually act on. O6 is a process objective — this is a mentorship project, and how the team works together is itself an evaluated outcome.

| ID | Objective | Definition of Done |
|---|---|---|
| 01 | Build a demand forecasting model | Produce daily-level forecasts of UnitsSold for a 30-day forward horizon, using historical patterns and exogenous regressors (marketing spend, competitor pricing, search interest, promotions, holidays). |
| 02 | Translate demand into inventory recommendations | Compute reorder points, safety stock, and order quantities dynamically, and flag stockout risk before it happens, using the demand forecast as the primary input. |
| 03 | Translate demand into workforce recommendations | Compute required headcount, flag understaffing/overtime risk, and recommend hiring timing that accounts for onboarding lag, using the same demand forecast as input. |
| 04 | Quantify the cost of getting it wrong | Build a cost model that expresses stockouts, excess holding cost, overtime cost, and backlog cost in monetary terms, so recommendations can be ranked by business impact, not just statistical accuracy. |
| 05 | Deliver an integrated decision-support dashboard | Present forecasts, inventory status, workforce status, and recommendations in a single interactive view, with support for at least one what-if scenario (e.g. "what if demand rises 20%"). |
| 06 | Demonstrate sound collaborative engineering practice | Use version control with a clear branch/PR history, modular code organized by pipeline stage, and a documented handoff between the two team members' areas of ownership. |

## 2.1 Explicit Non-Objectives

To keep scope realistic, the following are explicitly out of scope, and the team should say so confidently if asked, rather than attempting them under time pressure:

- Multi-product or multi-store forecasting (the architecture should be extensible to this, but building it is not required).
- Real-time/streaming data ingestion. All data is provided as static historical CSVs; the system is a batch-trained, batch-scored platform, not a live production service.
- Automated procurement or HR actions (e.g. the system recommends a reorder; it does not place a purchase order with a real supplier API).
- User authentication, multi-tenant support, or production-grade deployment infrastructure.

# 3. Expected Outcomes & Deliverables

By the end of the project, the team should have produced the following concrete artifacts. Each is independently demoable, which matters both for the final presentation and for de-risking the project — if one module runs behind schedule, the others should still stand on their own.

| Deliverable | Definition |
|---|---|
| **Forecasting notebook/module** | Trained model(s) for 30-day demand forecast, with back test evaluation (MAPE, RMSE) against a held-out period, and a feature importance / explainability view. |
| **Inventory planning module** | Code that computes rolling reorder point, safety stock, EOQ-based order quantity, and a stockout risk score per day, validated against the historical Inventory dataset. |
| **Workforce planning module** | Code that computes required headcount from the forecast, compares to a staffing plan, computes overtime/backlog exposure, and recommends hiring timing accounting for lag. |
| **Cost model** | A combined cost function expressing holding cost, stockout cost, labor cost, and overtime cost in INR, usable to compare scenarios. |
| **Integrated dashboard** | A Streamlit (or equivalent) app showing: demand forecast chart, inventory status panel, workforce status panel, and at least one interactive what-if control. |
| **Project documentation** | README, data dictionary (this document's Section 4), architecture diagram, and a model card describing each model's purpose, inputs, outputs, and known limitations. |
| **Final presentation & demo** | A live or recorded walkthrough covering the problem, approach, results, and at least one concrete example of a recommendation the system produced and why. |

## 3.1 Outcome Statement

**Target end-state**

Given a date, the system should be able to answer three questions in one view:

1) How many units of EcoFit Smart Bottle will likely sell in the next 30 days?
2) Given that, when should we reorder stock, how much, and what is our stockout risk?
3) Given that, do we have enough staff, and if not, when should hiring start?

Each answer should be backed by a number the team can defend, not a guess.

## 3.2 Success Metrics

- Forecast accuracy: MAPE under 15% on a held-out 60-day test window (a reasonable target for daily retail-style demand with the regressors provided).
- Inventory logic correctly flags at least 80% of the 16 historical stockout days in the Inventory dataset as "high risk" when run retrospectively.

- Workforce logic correctly flags at least 80% of the 47 historical backlog days in the Workforce dataset as "high risk" when run retrospectively.
- Dashboard loads and responds to a what-if input change in under 5 seconds.

# 4. Data Dictionary

All three datasets are synthetic, generated in Python (NumPy/Pandas), and span the same 730-day window (Jan 1, 2023 – Dec 30, 2024) for a single product, "EcoFit Smart Bottle." They share Date and Product as join keys. The three CSV files (synthetic_sales_data_v2.csv, synthetic_inventory_data_v2.csv, synthetic_workforce_data_v2.csv) are attached alongside this document.

## 4.1 Sales Dataset — synthetic_sales_data_v2.csv

*Role: upstream demand signal. Every other module derives its core input from this file's UnitsSold column.*

| Column | Type | Description | Range |
|---|---|---|---|
| `Date` | Date | Calendar date, one row per day. | 2023-01-01 to 2024-12-30 |
| `Product` | Text | Constant product name across the dataset. | "EcoFit Smart Bottle" |
| `DayOfWeek` | Text | Day name, derived from Date. | Monday … Sunday |
| `UnitsSold` | Float | Units sold that day — the core demand signal. | 0 - 441; 8 NaNs |
| `UnitPrice` | Float (INR) | Selling price per unit; 3 price regime changes. | 245 / 250 / 260 / 270 |
| `Revenue` | Float (INR) | UnitsSold x UnitPrice. | Derived |
| `MarketingSpend` | Float (INR) | Daily ad spend; affects demand with a 1-day lag. | ~2,000 - 22,600 |
| `CompetitorPriceIndex` | Float | 100 = price parity; below = competitor undercutting. | ~99 - 117 |
| `WebSearchInterest` | Float | Leading indicator; affects demand with a 2-day lag. | 10 - 100 |
| `IsPromo` | Binary | Promotional campaign flag. | 1 on ~8% of days |
| `IsHoliday` | Binary | Indian national holiday / festive window flag. | 1 on 18 days/yr |

## 4.2 Inventory Dataset — synthetic_inventory_data_v2.csv

*Role: downstream supply-chain response to demand. Simulates daily stock depletion, replenishment, and lead-time risk.*

| Column | Type | Description | Range |
|---|---|---|---|
| **Date** | Date | Join key, shared with Sales and Workforce. | Same range |
| **StockOnHand** | Integer | Physical units in warehouse at end of day. | 0 - ~4,000 |
| **UnitsSold** | Float | Carried over from Sales for traceability. | Identical to Sales |
| **ReorderPoint** | Integer | Rolling-demand-based reorder trigger level. | ~900 - 1,500 |
| **SafetyStock** | Integer | Buffer at 95% service level (z=1.65). | ~55 - 100 |
| **OrderPlaced** | Binary | New purchase order triggered that day. | 1 on 51 days |
| **OrderQuantity** | Integer | EOQ-based order size. | ~2,800 - 4,000 |
| **SupplierLeadTimeDays** | Integer | Days to receive an order; 5 disruption events. | 2 - 17 days |
| **OrderArrived** | Binary | Order received into stock that day. | 1 on arrival days |
| **StockoutFlag** | Binary | Stock hit zero, demand unmet. | 1 on 16 days |
| **UnitsShort** | Integer | Lost-sales units on stockout days. | 0 except stockouts |
| **DailyHoldingCost** | Float (INR) | Cost of holding that day's stock. | Derived |

## 4.3 Workforce Dataset — synthetic_workforce_data_v2.csv

*Role: downstream labor response to demand. Simulates required vs. staffed headcount, hiring lag, attrition, and overtime/backlog.*

| Column | Type | Description | Range |
|---|---|---|---|
| **Date** | Date | Join key, shared with Sales and Inventory. | Same range |
| **UnitsSold** | Float | Carried over from Sales for traceability. | Identical to Sales |
| **RequiredHeadcount** | Integer | Workers needed at 25 units/worker/day. | 6 - 12 |
| **StaffedHeadcount** | Integer | Actual workers on staff that day. | 5 - 12 |
| **HeadcountGap** | Integer | Required minus Staffed. | -4 to +6 |
| **OvertimeHoursPerWorker** | Float | OT used to cover shortfall, capped at 4 hrs. | 0.0 - 4.0 |
| **BacklogUnits** | Integer | Workload unprocessed even with OT. | 0 except 47 days |
| **BacklogFlag** | Binary | Processing backlog occurred. | 1 on 47 days |
| **EmployeeSatisfactionIndex** | Float | Declines under sustained OT, recovers when low. | 30 - 95 |
| **TotalLaborCost** | Float (INR) | Standard wage + 1.5x OT pay. | Derived |

# 5. How the Synthetic Datasets Are Used in This Project

This section is the practical bridge between the data dictionary (what each column means) and the methodology (how each module is built). It states explicitly which dataset feeds which module, and in what role — as a model's training data, as ground truth for validation, or as a live data source for the dashboard.

| Dataset | Used In | Role |
|---|---|---|
| `Sales` | Demand Forecasting | Primary modeling target. UnitsSold is the label; all other Sales columns (MarketingSpend, CompetitorPriceIndex, WebSearchInterest, IsPromo, IsHoliday, DayOfWeek) are candidate features. |
| `Sales` | Inventory Planning | UnitsSold (or its forecast) feeds the reorder point and safety stock formulas. The 8 NaN days must be handled (impute or exclude) before use here. |
| `Sales` | Workforce Planning | UnitsSold (or its forecast) feeds the required headcount calculation via the throughput constant. |
| `Inventory` | Inventory Planning | Ground truth for validating the team's own reorder-point/safety-stock logic. The historical StockoutFlag/UnitsShort columns let the team retrospectively test: "would our model have predicted this stockout?" |
| `Inventory` | Dashboard | StockOnHand, ReorderPoint, and SupplierLeadTimeDays populate the live inventory status panel. |
| `Workforce` | Workforce Planning | Ground truth for validating headcount/overtime logic, the same way Inventory validates stock logic. BacklogFlag lets the team test: "would our model have flagged this understaffing in advance?" |
| `Workforce` | Dashboard | RequiredHeadcount, StaffedHeadcount, and OvertimeHoursPerWorker populate the live workforce status panel. |
| `All three` | Cost Model | DailyHoldingCost, UnitsShort (lost-sale value via UnitPrice), TotalLaborCost, and OvertimeHoursPerWorker are combined into a single comparable cost figure per scenario. |

## 5.1 A Critical Distinction: Forecast vs. Historical Ground Truth

The team must keep two uses of these datasets conceptually separate, because conflating them is the most common mistake in a project like this:

1. Training & backtesting — the Sales dataset's historical UnitsSold is used to train a forecasting model and evaluate it on a held-out historical period (e.g. the last 60 days). This answers: "how good is our model at predicting demand we've already observed?"
2. Forward planning — once trained, the model produces a forecast for days beyond the dataset (e.g. the next 30 days past Dec 30, 2024). The Inventory and Workforce modules should be built to consume either historical actuals or this forward forecast through the same interface, so the same reorder-point and headcount logic works in both backtest mode and live planning mode.

## 5.2 Data Preparation Requirements Before Modeling

Before any model is trained, the team is expected to perform and document the following data preparation steps — these are deliverable-relevant, not optional cleanup:

- Identify and handle the 8 missing UnitsSold values in the Sales dataset (e.g. interpolation, forward-fill, or model-based imputation) — and justify the choice.
- Identify the 7 injected anomalies in Sales (4 stockout-driven zero/near-zero days, 3 viral demand spikes) and decide whether to exclude them from training, cap them, or model them explicitly as outliers. This decision should be stated, not silently made.
- Establish the correct lag structure for MarketingSpend (1-day lag) and WebSearchInterest (2-day lag) — these relationships exist in the data but are not labeled as "lagged" in the column names, and discovering this through correlation analysis at different lags is an expected EDA step.
- Cross-reference the 5 supply disruption windows in the Inventory dataset and the 9 attrition events in the Workforce dataset against the same dates in the Sales dataset, to build the narrative of cause and effect that the final presentation should tell.

# 6. Methodology by Module

This section defines the step-by-step approach for each of the three functional modules. The steps are sequential within a module but the three modules can be developed in parallel once the forecasting module produces its first usable output, since both downstream modules consume the forecast.

## 6.1 Module A — Demand Forecasting

*Objective: produce a reliable 30-day forward forecast of UnitsSold, with calibrated uncertainty, that the Inventory and Workforce modules can consume.*

| Step | What To Do |
|---|---|
| **1. EDA** | Decompose UnitsSold into trend, weekly seasonality, and yearly seasonality. Run lag-correlation analysis (lags 0-5 days) against MarketingSpend and WebSearchInterest to discover the true 1-day and 2-day lag relationships. Visualize the 7 injected anomalies. |
| **2. Feature engineering** | Construct lag features (MarketingSpend_lag1, WebSearchInterest_lag2), calendar features (day-of-week, month, IsHoliday, IsPromo), and rolling statistics (7-day and 30-day rolling mean/std of UnitsSold) as model inputs. |
| **3. Train/test split** | Time-based split, not random — train on the first ~22 months, hold out the last ~60 days as test. Never shuffle a time series split. |
| **4. Baseline model** | A naive seasonal baseline (e.g. "same day last week") and/or a simple moving average, established first so later models have something concrete to beat. |
| **5. Candidate models** | Facebook Prophet (with regressors) and XGBoost/LightGBM (with engineered lag and calendar features) as the two primary candidates; compare against the baseline. |
| **6. Evaluation** | MAPE and RMSE on the held-out test window. Target: MAPE under 15%. Also visually inspect forecast vs. actual on the 3 viral-spike anomaly days — the model is not expected to predict these, but should recover quickly afterward. |
| **7. Explainability** | SHAP values (for the tree-based model) or Prophet's built-in component plots, to show which features/seasonal components drive the forecast — this becomes a key dashboard panel. |

## 6.2 Module B — Inventory Optimization

*Objective: convert the demand forecast into actionable reorder timing, quantity, and stockout risk.*

| Step | What To Do |
|---|---|
| **1. Reorder point logic** | Implement ROP = (rolling average daily demand x average lead time) + safety stock, using the same formula structure as the synthetic data generator, but driven by the team's own forecast rather than the original generator's internal rolling average. |
| **2. Safety stock logic** | Implement safety stock = z x demand_std_dev x sqrt(lead_time), with z chosen for a target service level (95% -> z=1.65, consistent with how the dataset itself was built). |
| **3. EOQ / order quantity logic** | Implement the Economic Order Quantity formula using an assumed ordering cost and holding cost (the same INR 500/order and INR 8/unit/year used to build the dataset are reasonable defaults, but the team should treat these as configurable parameters). |
| **4. Stockout risk scoring** | For each forecasted day, compute the probability (or a risk tier: Low/Medium/High) that projected stock will fall below zero before the next order arrives, given forecast uncertainty and lead-time variability. |
| **5. Backtesting against ground truth** | Run the team's reorder-point and safety-stock logic retrospectively against the historical Inventory dataset's actual StockOnHand and SupplierLeadTimeDays, and check whether it would have flagged the 16 historical stockout days as high-risk in advance. Target: catch at least 80% of them. |
| **6. Lead time variability handling** | Treat SupplierLeadTimeDays not as a constant but as a distribution; the model should widen safety stock recommendations when recent lead times have been more volatile (e.g. during one of the 5 disruption windows). |

## 6.3 Module C — Workforce Planning

*Objective: convert the demand forecast into actionable hiring timing, overtime exposure, and backlog risk.*

| Step | What To Do |
|---|---|
| **1. Workload-to-headcount conversion** | Implement RequiredHeadcount = ceil(forecasted_units / units_per_worker_per_day), using a forward-looking smoothing window (e.g. 7-day average) consistent with how the dataset itself models planning behavior — ops teams plan against a trend, not single-day noise. |
| **2. Staffing gap detection** | Compute HeadcountGap = RequiredHeadcount - StaffedHeadcount for each forecasted day, using the team's actual current staffing level as the starting point (not the synthetic dataset's internal simulation, which was only used to generate realistic historical ground truth). |
| **3. Hiring lag-aware recommendation** | Because a new hire takes time to onboard (modeled in the dataset as a 10-day lag), the recommendation engine must recommend a hiring decision 10+ days BEFORE the projected shortfall, not on the day the shortfall appears. This is one of the most important behaviors to get right and to demo clearly. |
| **4. Overtime and backlog estimation** | When a forecasted shortfall is too close in time to correct via hiring, estimate how many overtime hours would be needed to cover it (capped at a configurable max, e.g. 4 hrs/day), and flag any residual shortfall as projected backlog. |
| **5. Attrition risk awareness (optional/stretch)** | If time permits, use the EmployeeSatisfactionIndex pattern (declining under sustained overtime) as a simple proxy signal for elevated attrition risk, and surface a qualitative warning when sustained OT is forecasted (this mirrors the dataset's own internal mechanic, where attrition is independent of planning but more contextually likely under strain). |
| **6. Backtesting against ground truth** | Run the team's headcount/overtime logic retrospectively against the historical Workforce dataset and check whether it would have flagged the 47 historical backlog days as high-risk in advance, with attention to the 2 explainable clusters (April-May ramp-up, February pre-hiring-wave). Target: catch at least 80%. |

## 6.4 Module D — Integrated Cost Model

*Objective: express the output of Modules B and C in comparable monetary terms, so different policies (e.g. "order more often in smaller batches" vs. "order less often in larger batches") can be ranked.*

| Cost Component | Computed From | Definition |
|---|---|---|
| **Holding cost** | DailyHoldingCost | Sum of daily holding cost over the planning horizon under a given inventory policy. |
| **Stockout / lost sales cost** | UnitsShort x UnitPrice | Revenue value of unmet demand under a given inventory policy. |
| **Standard labor cost** | StaffedHeadcount x daily wage | Base labor cost under a given staffing policy. |
| **Overtime cost** | OvertimeHoursPerWorker x hourly rate x 1.5 | Premium labor cost incurred to cover shortfalls. |
| **Backlog cost (optional)** | BacklogUnits x an assumed delay penalty | Optional: assign a cost to unprocessed workload if the team wants to compare backlog against overtime as alternatives. |

The combined cost model allows the team to answer questions like: "Is it cheaper to carry 10% more safety stock, or to risk occasional overtime?" — this kind of tradeoff analysis is a strong differentiator for the final presentation.

## 6.5 Module E — Integrated Dashboard

*Objective: present the outputs of all four modules above in one interactive view.*

- Panel 1 — Demand forecast chart with confidence interval, actual vs. forecast overlay, and a feature-importance/explainability side panel.
- Panel 2 — Inventory status: current stock, reorder point, days until projected stockout, recommended order quantity and timing.
- Panel 3 — Workforce status: current staffing, required headcount trend, recommended hiring timing, projected overtime/backlog exposure.
- Panel 4 — Cost comparison: side-by-side cost estimate for at least two scenarios (e.g. current policy vs. an adjusted policy).
- Interactive control — at minimum, a slider or input that lets the user simulate a demand shock (e.g. +20% demand for the next 14 days) and see Panels 2-4 update accordingly.

*Note: If Module E becomes a stretch for the team to achieve, the group can decide to park it from the MVP (minimum viable product).*

# 7. Machine Learning Models, Algorithms & Techniques

This section specifies which models and algorithms apply to which part of the system, and explains the reasoning so the team understands not just what to use but why. A recurring theme: machine learning is used where the problem is genuinely a prediction problem under uncertainty (forecasting future demand). Where the problem is a known, well-defined operations-research calculation (reorder point, safety stock, headcount conversion), the correct approach is the established formula, not a model — using ML where a formula suffices would be both unnecessary and harder to explain or trust.

| Model / Technique | Used For | Why This Model | Notes |
|---|---|---|---|
| **Facebook Prophet** | Demand Forecasting (primary) | Built-in handling of trend, weekly/yearly seasonality, and holiday effects; supports extra regressors (MarketingSpend_lag1, WebSearchInterest_lag2, CompetitorPriceIndex); produces calibrated uncertainty intervals out of the box, which the Inventory module's risk scoring depends on. | Component plots (trend/seasonality/regressor effect) are a free, strong explainability deliverable. |
| **XGBoost / LightGBM** | Demand Forecasting (comparison candidate) | Gradient-boosted trees on engineered lag, rolling, and calendar features; typically strong on tabular data with nonlinear regressor interactions (e.g. promo x holiday). | Use SHAP for feature importance; good complement to Prophet for the team to compare and discuss tradeoffs in the presentation. |
| **Naive seasonal baseline** | Demand Forecasting (baseline) | "Forecast = same day last week" or a simple moving average. Establishes the floor that Prophet/XGBoost must beat to justify their complexity. | Required, not optional — a model is only "good" relative to a stated baseline. |
| **Rule-based formulas (ROP, Safety Stock, EOQ)** | Inventory Optimization | Classical operations-research formulas (see Section 6.2), parameterized by the forecast's mean and uncertainty rather than hardcoded historical averages. | Not a black-box ML model — and that's appropriate. Inventory theory here is well-established analytically; ML is used upstream (the forecast) and downstream (risk classification), not to replace these formulas. |
| **Logistic Regression or Random Forest classifier (optional)** | Inventory Optimization — stockout risk scoring | If the team wants a probabilistic stockout risk score rather than a deterministic threshold check, a simple classifier trained on (days_until_next_order_arrival, current_stock, forecast_uncertainty) -> stockout_occurred (from the historical Inventory dataset) can output a calibrated probability. | Optional enhancement; the deterministic threshold approach in Section 6.2 Step 4 is sufficient for the MVP. |

| Model / Technique | Used For | Why This Model | Notes |
|---|---|---|---|
| **Rule-based formulas (headcount conversion)** | Workforce Planning | RequiredHeadcount = ceil(forecast / throughput_constant), smoothed — same reasoning as the inventory formulas: this is a deterministic operations calculation downstream of an ML forecast, not itself a model to be trained. | Throughput constant should be a configurable parameter, ideally estimated from the historical Workforce dataset (UnitsSold / StaffedHeadcount on well-staffed days) rather than assumed. |
| **Anomaly detection — Z-score or Isolation Forest (optional)** | Sales EDA / data cleaning | Useful for systematically identifying the 7 injected anomalies (and any the team might introduce when extending the dataset) rather than finding them by visual inspection alone. | Good for demonstrating a more rigorous EDA process; not required since the anomalies are documented in this spec, but encouraged as good practice. |

## 7.1 Evaluation Metrics

| Metric | Applies To | Purpose |
|---|---|---|
| **MAPE (Mean Absolute Percentage Error)** | Forecasting | Primary accuracy metric; target under 15% on the held-out test window. Intuitive for non-technical stakeholders ("on average, off by X%"). |
| **RMSE (Root Mean Squared Error)** | Forecasting | Secondary metric; penalizes large misses more heavily, useful for understanding how the model handles the 3 viral-spike anomaly days specifically. |
| **Precision / Recall on risk flags** | Inventory & Workforce | For the stockout-risk and backlog-risk flags: precision (when we flag risk, how often were we right?) and recall (of all actual historical stockouts/backlogs, how many did we flag in advance?). Target recall ≥ 80% per Section 3.2. |
| **Total cost under policy** | Cost Model | The ultimate business-facing metric — not a statistical metric but a financial one, used to compare alternative inventory/staffing policies against each other. |

## 7.2 Model Selection Guidance

The team should not pick a single model on day one and commit blindly. The recommended path is:

3. Build the naive baseline first. This takes under a day and gives an immediate floor to beat.
4. Build Prophet with regressors second. It is fast to set up, handles seasonality and holidays natively, and gives interpretable component plots — strong for both speed and explainability.
5. Build XGBoost/LightGBM third, using the lag and rolling features from Section 6.1. Compare its test-set MAPE against Prophet's.
6. Choose whichever model wins on the held-out test window as the dashboard's production model, but keep both in the repository and present the comparison in the final demo — "we tried two approaches and here's why we chose this one" is a stronger story than presenting only one model with no comparison.

## 7.3 Handling Forecast Uncertainty Downstream

A forecast is never a single number in a real planning system — it has a confidence interval. This matters concretely for Module B and Module C:

- Safety stock should widen when the forecast's uncertainty interval is wider (e.g. heading into a season with less historical precedent, like the first summer peak in the dataset).
- Workforce hiring recommendations should be more conservative (recommend hiring slightly earlier or for a slightly higher headcount) when forecast uncertainty is high, since under-hiring risk (backlog) and over-hiring risk (idle labor cost) are asymmetric costs the team can quantify via Module D.

**Common pitfall to avoid**

Do not pass only the forecast's point estimate (the mean) downstream and discard the uncertainty interval. This is the single most common shortcut that makes a forecasting-to-planning pipeline look naive in a demo. Prophet and most quantile-based gradient boosting setups both produce interval estimates with little extra effort — use them.

# 8. System Design & Architecture

## 8.1 High-Level Architecture

The system follows a simple, linear data flow with two parallel downstream branches, matching the dataset architecture described in the earlier Data Dictionary & Dataset Connections reference:

**Architecture at a glance**

Historical Sales Data -> Data Cleaning & Feature Engineering -> Demand Forecasting Model -> [forecast output, with uncertainty] -> splits into two parallel branches:
(A) Inventory Optimization Module -> reorder/stockout recommendations, and
(B) Workforce Planning Module -> hiring/overtime recommendations.

Both branches feed into the Cost Model, and all outputs converge in the Integrated Dashboard.

## 8.2 Technology Stack

| Layer | Technology | Rationale |
|---|---|---|
| **Data processing** | Pandas, NumPy | All three datasets are CSV; no database required for this scope. |
| **Forecasting** | Prophet, XGBoost / LightGBM, scikit-learn | Prophet for the primary model; XGBoost for comparison and SHAP-based explainability. |
| **Inventory & Workforce logic** | Pure Python / Pandas (formula-based, see Section 6) | No specialized ML library needed; these are deterministic calculations parameterized by the forecast. |
| **Backend (optional, if dashboard needs an API layer)** | FastAPI | Only needed if the dashboard and modeling code run as separate services; for a 10-week student project, a monolithic Streamlit app calling Python functions directly is simpler and acceptable. |
| **Dashboard (optional for MVP)** | Streamlit (recommended) or Plotly Dash | Both support the charting and interactivity needs in Section 6.5 with minimal boilerplate. |
| **Visualization (optional for MVP)** | Plotly | Interactive charts for forecast, inventory, and workforce panels. |
| **Version control (optional for MVP)** | Git + GitHub | Feature-branch workflow; see Section 8.3 for the expected collaboration pattern. |

## 8.3 Suggested Team Ownership & Collaboration Pattern

Given the architecture's natural split into a supply-chain branch and a labor branch, the two team members can divide ownership while still collaborating on the shared forecasting module and final dashboard:

- Student A — owns Module A (Demand Forecasting) and Module B (Inventory Optimization).

- Student B — owns Module C (Workforce Planning) and Module E (Integrated Dashboard), with Module D (Cost Model) built jointly since it draws from both branches.
- Both — jointly review the forecasting module's output interface early, since both downstream modules depend on its shape - agreeing on this interface early prevents integration pain later.

Recommended Git workflow: a shared main branch protected from direct pushes, feature branches per module (e.g. feature/forecasting-prophet, feature/inventory-rop), and pull requests reviewed by the other team member before merging — even on a 2-person team, this creates a real code review habit and a clean commit history that is itself part of the O6 objective (Section 2).

## 8.4 Module Interfaces

To keep Module A's output usable by both downstream modules without tight coupling, the forecasting module should expose a simple, stable interface, for example a function that returns a dataframe with columns: Date, ForecastUnits, ForecastLower, ForecastUpper. Both the Inventory and Workforce modules should be written to consume this shape, whether it comes from Prophet, XGBoost, or — during early development — directly from the historical Sales dataset's actual UnitsSold (so Modules B and C can be built and tested before Module A is finished).

**Why this interface decision matters**

Because Modules B and C only depend on this dataframe shape, the team can develop all three modules in parallel from Week 1: Module A's owner builds the real forecasting model, while Module B and C's owners build and test their logic against the actual historical UnitsSold column (treating it as a stand-in "perfect forecast"). Once Module A is ready, swapping in the real forecast requires no changes downstream — this is the single most important design decision for hitting the 10-week timeline.

# 9. Suggested Timeline & Acceptance Criteria

## 9.1 Week-by-Week Plan

| Week(s) | Activity | Primary Owner |
|---|---|---|
| 1-2 | Project setup; EDA on all 3 datasets; data cleaning decisions documented; naive baseline forecast built. | Both |
| 3-4 | Prophet and XGBoost forecasting models built and backtested; forecast output interface finalized. | Student A (lead), Student B (consumes interface) |
| 5-6 | Inventory module (ROP, safety stock, EOQ, stockout risk) and Workforce module (headcount, OT, backlog risk) built in parallel against the agreed forecast interface; backtested against historical Inventory/Workforce datasets. | Student A: Inventory / Student B: Workforce |
| 7 | Cost model built; modules integrated; dashboard skeleton with all 4 panels wired to real outputs. | Both |
| 8 | What-if interactivity added to dashboard; explainability panel (SHAP/Prophet components) added; backtesting accuracy targets validated against Section 3.2. | Both |
| 9 | End-to-end testing; edge cases (missing data days, anomaly days) verified; documentation (README, model card) written. | Both |
| 10 | Final polish, rehearsal, and presentation delivery. | Both |

## 9.2 Acceptance Criteria

These are the concrete, checkable conditions the mentor will use to assess whether each module meets the objectives in Section 2. They map directly to the success metrics in Section 3.2.

| Module | Acceptance Condition | Evidence Required |
|---|---|---|
| Forecasting | MAPE < 15% on held-out 60-day test window; baseline comparison shown; explainability view present. | Backtest report + dashboard panel |
| Inventory | ≥ 80% of the 16 historical stockout days flagged as high-risk retrospectively; reorder point and EOQ logic implemented per Section 6.2. | Backtest report + dashboard panel |
| Workforce | ≥ 80% of the 47 historical backlog days flagged as high-risk retrospectively; hiring recommendation correctly precedes shortfall by ≥ lead time. | Backtest report + dashboard panel |
| Cost Model | All 4 cost components (holding, stockout, labor, overtime) computed and combinable into one comparable total per scenario. | Dashboard cost comparison panel |
| Dashboard (optional) | All 4 panels functional; what-if control updates Panels 2-4 in under 5 seconds; runs without crashing on the full 730-day dataset. | Live demo |

| Module | Acceptance Condition | Evidence Required |
|---|---|---|
| `Documentation (optional)` | README, data dictionary, architecture diagram, and model card all present and accurate as of final submission. | Repository review |
| `Collaboration (optional)` | Git history shows a clear feature-branch/PR pattern from both contributors; ownership split is visible and roughly balanced. | Repository review |

## 9.3 Risk Register

- Risk: forecasting model underperforms the 15% MAPE target. Mitigation: the baseline and backtest harness should be built early into the project, so underperformance is visible early, with limited time remaining to iterate on features or try an alternative model.
- Risk: the two downstream modules get blocked waiting for the forecasting module. Mitigation: the module interface decision in Section 8.4 explicitly allows Modules B and C to start immediately using historical UnitsSold as a stand-in, decoupling the timeline.
- Risk: dashboard integration reveals incompatible assumptions between modules built independently. Mitigation: the interface review checkpoint in Section 8.3 is designed to catch this before it becomes a high risk.